# Namespace TUIS

## Classes

Terminal

A Terminal is the main object of this TUI system. It will hold each Component that will be drawn on the screen.

## Enums

Terminal.BackgroundColor

Get the Terminal.BackgroundColor using the ANSI color code.

Terminal.TextColor

Get the Terminal.TextColor using th ANSI color code.

Terminal.TextDecoration

Get the Terminal.TextDecoration using the ANSI color code.

# Class Terminal

Namespace: TUIS

Assembly: TUIS.dll

A Terminal is the main object of this TUI system. It will hold each Component that will be drawn on the screen.

```
public class Terminal
```

**Inheritance**

object ← Terminal

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Terminal(int, int)

```
public Terminal(int width, int height)
```

#### Parameters

`width` int

`height` int

## Fields

### Components

```
public readonly List<Component> Components
```

### Field Value

List<Component>

## Height

```
public readonly int Height
```

### Field Value

int

## InputSystem

```
public readonly InputSystem InputSystem
```

### Field Value

InputSystem

## NeedReDraw

```
public readonly (int min, int max)[] NeedReDraw
```

### Field Value

(int min, int max)[]

## TimeSystem

```
public readonly TimeSystem TimeSystem
```

### Field Value

TimeSystem

## Width

```
public readonly int Width
```

### Field Value

int

# Methods

## Clear()

```
public void Clear()
```

## DrawChar(int, int, char, TextColor, BackgroundColor, TextDecoration)

This method will be called by DrawChar(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?) to correctly update a character of the internal TUIS.Terminal._screen of the Terminal.

```
public void DrawChar(int y, int x, char c, Terminal.TextColor textColor,
Terminal.BackgroundColor backgroundColor, Terminal.TextDecoration textDecoration)
```

### Parameters

y int

The y coordinate of the char to update (0 = top).

x int

The x coordinate of the char to update (0 = left).

c char

The new char.

`textColor` Terminal.TextColor

The Terminal.TextColor the c will be drawn.

`backgroundColor` Terminal.BackgroundColor

The Terminal.BackgroundColor the c will be drawn.

`textDecoration` Terminal.TextDecoration

The Terminal.TextDecoration the c will be drawn.

## Start(Action<Terminal>?)

```
public void Start(Action<Terminal>? onStart = null)
```

### Parameters

`onStart` Action<Terminal>

# Enum Terminal.BackgroundColor

Namespace: TUIS

Assembly: TUIS.dll

Get the Terminal.BackgroundColor using the ANSI color code.

```
public enum Terminal.BackgroundColor
```

## Fields

`Black = 40`

`Blue = 44`

`Cyan = 46`

`Green = 42`

`None = 0`

`Purple = 45`

`Red = 41`

`White = 47`

`Yellow = 43`

# Enum Terminal.TextColor

Namespace: TUIS

Assembly: TUIS.dll

Get the Terminal.TextColor using th ANSI color code.

```
public enum Terminal.TextColor
```

## Fields

`Black = 30`

`Blue = 34`

`Cyan = 36`

`Green = 32`

`Purple = 35`

`Red = 31`

`White = 37`

`Yellow = 33`

# Enum Terminal.TextDecoration

Namespace: TUIS

Assembly: TUIS.dll

Get the Terminal.TextDecoration using the ANSI color code.

```
public enum Terminal.TextDecoration
```

## Fields

`Bold = 1`

`Default = 0`

`Underline = 4`

# Namespace TUIS.Components

## Classes

Component

# Class Component

Namespace: TUIS.Components

Assembly: TUIS.dll

```
public class Component
```

**Inheritance**

object ← Component

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## Component(Terminal, int, int, int, int)

A Component will represent an element drawn on the Terminal.

```
public Component(Terminal terminal, int width, int height, int posY, int posX)
```

## Parameters

`terminal` Terminal

The Terminal the Component will be attached to.

`width` int

The width of the component (-1 = Width).

`height` int

The height of the component (-1 = Height).

`posY` int

The position on the y axis of the top left most char of the component (0 = top, -1 = will ceter the component).

`posX` int

The position on the x axis of the top left most char of the component (0 = left, -1 = will ceter the component).

# Fields

## Masks

```
public readonly List<Mask> Masks
```

### Field Value

List<Mask>

## Terminal

```
public Terminal Terminal
```

### Field Value

Terminal

# Properties

## Height

```
public int Height { get; set; }
```

### Property Value

int

## IsVisible

```
public bool IsVisible { get; set; }
```

### Property Value

bool

## NeedReDraw

```
public bool NeedReDraw { get; set; }
```

### Property Value

bool

## PosX

```
public int PosX { get; set; }
```

### Property Value

int

## PosY

```
public int PosY { get; set; }
```

### Property Value

int

## Width

```
public int Width { get; set; }
```

### Property Value

int

## Methods

### Draw()

```
public void Draw()
```

# Namespace TUIS.Components.Masks

## Classes

BackgroundMask

Print a single char in the background (Warning Masks will be printed in order so this one will override any Mask before itself)

BoxMask

This Mask will draw a box on the border of the Component.

ClockMask

This Mask will draw a clock and updates it accordingly to the real time.

CustomMask

This Mask helps to create a Mask with unique behaviour without needing to create an entire new class.

ImageMask

This Mask will draw an image in format jpg, png or any other image format in ASCII art in black and white or using 8 different colors.

InputMask

WIP. This mask let you use ReadLine() to get some user input as command or text

Mask

A Mask must be attached to a Component and adds a layer of drawings on top of it. Masks will be drawn in the same order as they have been added to the Masks list.

TextMask

This Mask will draw literal custom text.

## Enums

BoxMask.Type

The Type of boxing

TextMask.HorizontalAlignmentEnum

TextMask.VerticalAlignmentEnum

# Class BackgroundMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

Print a single char in the background (Warning Masks will be printed in order so this one will override any Mask before itself)

```
public class BackgroundMask : Mask
```

**Inheritance**

object ← Mask ← BackgroundMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### BackgroundMask(Component, char, bool, TextColor, BackgroundColor, TextDecoration)

```
public BackgroundMask(Component component, char backgroundChar = ' ', bool isVisible
= true, Terminal.TextColor color = TextColor.White, Terminal.BackgroundColor
background = BackgroundColor.None, Terminal.TextDecoration decoration
= TextDecoration.Default)
```

#### Parameters

`component` Component

`backgroundChar` char

`isVisible` bool

`color` Terminal.TextColor

`background` Terminal.BackgroundColor

`decoration` Terminal.TextDecoration

# Properties

## BackgroundChar

Holds the character to be drawn as the background.

```
public char BackgroundChar { get; set; }
```

### Property Value

char

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

This Mask will draw the BackgroundChar on top of the intire Component.

```
protected override void Behaviour()
```

# Class BoxMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

This Mask will draw a box on the border of the Component.

```
public class BoxMask : Mask
```

**Inheritance**

object ← Mask ← BoxMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### BoxMask(Component, Type, bool, TextColor, BackgroundColor, TextDecoration)

Add a bounding box on the component.

```
public BoxMask(Component component, BoxMask.Type type, bool isVisible = true,
Terminal.TextColor color = TextColor.White, Terminal.BackgroundColor background =
BackgroundColor.None, Terminal.TextDecoration decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

`type` BoxMask.Type

The BoxMask.Type of boxing.

`isVisible` bool

`color` Terminal.TextColor

`background` Terminal.BackgroundColor

`decoration` Terminal.TextDecoration

# Properties

## BoxType

Holds the BoxMask.Type of boxing.

```
public BoxMask.Type BoxType { get; set; }
```

### Property Value

BoxMask.Type

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

This Mask will draw a box on the border of the Component.

```
protected override void Behaviour()
```

# Enum BoxMask.Type

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

The Type of boxing

```
public enum BoxMask.Type
```

## Fields

`Bold = 2`

`Default = 0`

`Double = 3`

`None = 6`

`Rounded = 1`

`Thick = 4`

`ThickInner = 5`

# Class ClockMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

This Mask will draw a clock and updates it accordingly to the real time.

```
public class ClockMask : Mask
```

**Inheritance**

object ← Mask ← ClockMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### ClockMask(Component, bool, TextColor, BackgroundColor, TextDecoration)

Add a real time clock to the Component. Warning: The clock is 27 wide by 3 tall. Any Component smaller than that will cause the clock to overshoot.

```
public ClockMask(Component component, bool isVisible = true, Terminal.TextColor
color = TextColor.White, Terminal.BackgroundColor background = BackgroundColor.None,
Terminal.TextDecoration decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

The component which the mask is attached to.

`isVisible` bool

Represent the visibility of the mask (default = true).

`color` Terminal.TextColor

The default color of the mask (a mask's Behaviour()) method can override the color (default = white).

`background` Terminal.BackgroundColor

The default background color of the mask (a mask's Behaviour()) method can override the background color (default = None).

`decoration` Terminal.TextDecoration

The default decoration of the mask (a mask's Behaviour()) method can override the decoration (default = Default).

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

This Mask's Behaviour() is to display large numbers .

```
protected override void Behaviour()
```

# Class CustomMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

This Mask helps to create a Mask with unique behaviour without needing to create an entire new class.

```
public class CustomMask : Mask
```

**Inheritance**

object ← Mask ← CustomMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### CustomMask(Component, Action<CustomMask>, bool, TextColor, BackgroundColor, TextDecoration)

Create a Mask with unique Behaviour() (`action`).

```
public CustomMask(Component component, Action<CustomMask> action, bool isVisible
= true, Terminal.TextColor color = TextColor.White, Terminal.BackgroundColor
background = BackgroundColor.None, Terminal.TextDecoration decoration
= TextDecoration.Default)
```

#### Parameters

`component` Component

The component which the mask is attached to.

`action` Action<CustomMask>

The Action to perform when Behaviour() is called.

`isVisible` bool

Represent the visibility of the mask (default = true).

`color` Terminal.TextColor

The default color of the mask (a mask's Behaviour()) method can override the color (default = white).

`background` Terminal.BackgroundColor

The default background color of the mask (a mask's Behaviour()) method can override the background color (default = None).

`decoration` Terminal.TextDecoration

The default decoration of the mask (a mask's Behaviour()) method can override the decoration (default = Default).

# Fields

## Properties

Holds properties used in the TUIS.Components.Masks.CustomMask._action. (Useless for now).

```
public readonly Dictionary<string, object> Properties
```

## Field Value

Dictionary<string, object>

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

This Mask's Behaviour() is to call the TUIS.Components.Masks.CustomMask._action.

```
protected override void Behaviour()
```

# Class ImageMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

This Mask will draw an image in format jpg, png or any other image format in ASCII art in black and white or using 8 different colors.

```
public class ImageMask : Mask
```

**Inheritance**

object ← Mask ← ImageMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### ImageMask(Component, string, bool, bool, TextColor, BackgroundColor, TextDecoration)

Draw an image in format jpg, png or any other image format in ASCII art. Warning: the original image should have a ratio than the Component.

```
public ImageMask(Component component, string path, bool isColored = false, bool
isVisible = true, Terminal.TextColor color = TextColor.White,
Terminal.BackgroundColor background = BackgroundColor.None, Terminal.TextDecoration
decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

The component which the mask is attached to.

`path` string

The relative or absolute path of the image to draw is ASCII art

`isColored` bool

Whether the Image will be drawn using the 8 possible colors or in black and white (default = false).

`isVisible` bool

Represent the visibility of the mask (default = true).

`color` Terminal.TextColor

The default color of the mask (a mask's Behaviour()) method can override the color (default = white).

`background` Terminal.BackgroundColor

The default background color of the mask (a mask's Behaviour()) method can override the background color (default = None).

`decoration` Terminal.TextDecoration

The default decoration of the mask (a mask's Behaviour()) method can override the decoration (default = Default).

# Properties

## IsColored

Holds whether the image should be drawn in color or in black and white.

```
public bool IsColored { get; set; }
```

### Property Value

bool

# Methods

# Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

This Mask's Behaviour() is to draw the TUIS.Components.Masks.ImageMask._image in ascii art.

```
protected override void Behaviour()
```

# Export(string, int?, int?)

```
public void Export(string path = "./exported_image.png", int? exportedWidth = null,
int? exportedHeight = null)
```

## Parameters

`path` string

`exportedWidth` int?

`exportedHeight` int?

# Class InputMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

WIP. This mask let you use ReadLine() to get some user input as command or text

```
public class InputMask : Mask
```

**Inheritance**

object ← Mask ← InputMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### InputMask(Component, Action<InputMask>?, byte, byte, bool, TextColor, BackgroundColor, TextDecoration)

```
public InputMask(Component component, Action<InputMask>? onOutputChange = null, byte
horizontalPadding = 0, byte verticalPadding = 0, bool isVisible = true,
Terminal.TextColor color = TextColor.White, Terminal.BackgroundColor background =
BackgroundColor.None, Terminal.TextDecoration decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

`onOutputChange` Action<InputMask>

`horizontalPadding` byte

`verticalPadding` byte

`isVisible` bool

`color` Terminal.TextColor

`background` Terminal.BackgroundColor

`decoration` Terminal.TextDecoration

# Properties

## Enabled

Holds whether or not the Mask should read the user inputs.

```
public bool Enabled { get; set; }
```

### Property Value

bool

## HorizontalPadding

```
public byte HorizontalPadding { get; set; }
```

### Property Value

byte

## Output

Holds the string corresponding to the last input of the user

```
public string Output { get; }
```

### Property Value

string

## VerticalPadding

```
public byte VerticalPadding { get; set; }
```

### Property Value

byte

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

```
protected override void Behaviour()
```

# Class Mask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

A Mask must be attached to a Component and adds a layer of drawings on top of it. Masks will be drawn in the same order as they have been added to the Masks list.

```
public abstract class Mask
```

**Inheritance**

object ← Mask

**Derived**

BackgroundMask, BoxMask, ClockMask, CustomMask, ImageMask, InputMask, TextMask

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### Mask(Component, bool, TextColor, BackgroundColor, TextDecoration)

A Mask goes in a Component and will add characters to print on it following custom rules: Behaviour().

```
public Mask(Component component, bool isVisible = true, Terminal.TextColor color =
TextColor.White, Terminal.BackgroundColor background = BackgroundColor.None,
Terminal.TextDecoration decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

The component which the mask is attached to.

`isVisible` bool

Represent the visibility of the mask (default = true).

`color` Terminal.TextColor

The default color of the mask (a mask's Behaviour()) method can override the color (default = white).

`background` Terminal.BackgroundColor

The default background color of the mask (a mask's Behaviour()) method can override the background color (default = None).

`decoration` Terminal.TextDecoration

The default decoration of the mask (a mask's Behaviour()) method can override the decoration (default = Default).

# Fields

## Component

The Component the Mask is attached to.

```
public readonly Component Component
```

### Field Value

Component

# Properties

## Background

Keeps the current Terminal.BackgroundColor of the Mask.

```
public Terminal.BackgroundColor Background { get; set; }
```

### Property Value

Terminal.BackgroundColor

## Color

Keeps the current Terminal.TextColor of the Mask.

```
public Terminal.TextColor Color { get; set; }
```

### Property Value

Terminal.TextColor

## Decoration

Keeps the current Terminal.TextDecoration of the Mask.

```
public Terminal.TextDecoration Decoration { get; set; }
```

### Property Value

Terminal.TextDecoration

## IsVisible

Represent the visibility of the Mask.

```
public bool IsVisible { get; set; }
```

### Property Value

bool

## NeedReDraw

Helps to optimize the drawing of components, if NeedReDraw == false, the Mask won't be re-Draw()n. if NeedReDraw is set to true, it will automatically set the NeedReDraw to true.

```
public bool NeedReDraw { get; set; }
```

### Property Value

bool

## Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

```
protected abstract void Behaviour()
```

## Draw()

This method will be called if the mask has changed and is visible. It calls the Behaviour() method of the mask.

```
public void Draw()
```

## DrawChar(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?)

Used in the Behaviour() method of Masks to properly print a character using relative coordinates.

```
protected void DrawChar(int y, int x, char? c, Terminal.TextColor? textColor = null,
Terminal.BackgroundColor? backgroundColor = null, Terminal.TextDecoration?
textDecoration = null)
```

### Parameters

y int

Coordinate on the y-axis (0 = top) relative to the Component.

x int

Coordinate on the x-axis (0 = left) relative to the Component.

`c` char?

The character to print (null = no print).

`textColor` Terminal.TextColor?

The color which the `c` will be printed. null = the default background color of the mask.

`backgroundColor` Terminal.BackgroundColor?

The background color which the `c` will be printed. null = the default background color of the mask.

`textDecoration` Terminal.TextDecoration?

The decoration with which the `c` will be printed. null = the default decoration of the mask.

# Class TextMask

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

This Mask will draw literal custom text.

```
public class TextMask : Mask
```

**Inheritance**

object ← Mask ← TextMask

**Inherited Members**

Mask.Component , Mask.NeedReDraw , Mask.IsVisible , Mask.Color , Mask.Background , Mask.Decoration , Mask.Draw() , Mask.DrawChar(int, int, char?, Terminal.TextColor?, Terminal.BackgroundColor?, Terminal.TextDecoration?) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### TextMask(Component, string, byte, byte, HorizontalAlignmentEnum, VerticalAlignmentEnum, bool, TextColor, BackgroundColor, TextDecoration)

```
public TextMask(Component component, string text, byte horizontalPadding = 0, byte
verticalPadding = 0, TextMask.HorizontalAlignmentEnum horizontalAlignment =
HorizontalAlignmentEnum.Left, TextMask.VerticalAlignmentEnum verticalAlignment =
VerticalAlignmentEnum.Top, bool isVisible = true, Terminal.TextColor color =
TextColor.White, Terminal.BackgroundColor background = BackgroundColor.None,
Terminal.TextDecoration decoration = TextDecoration.Default)
```

#### Parameters

`component` Component

`text` string

`horizontalPadding` byte

`verticalPadding` byte

`horizontalAlignment` TextMask.HorizontalAlignmentEnum

`verticalAlignment` TextMask.VerticalAlignmentEnum

`isVisible` bool

`color` Terminal.TextColor

`background` Terminal.BackgroundColor

`decoration` Terminal.TextDecoration

# Properties

## HorizontalAlignment

```
public TextMask.HorizontalAlignmentEnum HorizontalAlignment { get; set; }
```

### Property Value

TextMask.HorizontalAlignmentEnum

## HorizontalPadding

```
public byte HorizontalPadding { get; set; }
```

### Property Value

byte

## Text

```
public string Text { get; set; }
```

### Property Value

string

## VerticalAlignment

```
public TextMask.VerticalAlignmentEnum VerticalAlignment { get; set; }
```

### Property Value

TextMask.VerticalAlignmentEnum

## VerticalPadding

```
public byte VerticalPadding { get; set; }
```

### Property Value

byte

# Methods

## Behaviour()

This method is what makes the mask unique, it will describe that to print and where using the Draw Char(int, int, char?, TextColor?, BackgroundColor?, TextDecoration?). The method will automatically be called when needed.

```
protected override void Behaviour()
```

# Enum TextMask.HorizontalAlignmentEnum

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

```
public enum TextMask.HorizontalAlignmentEnum
```

## Fields

`Center = 1`

`Left = 0`

`Right = 2`

# Enum TextMask.VerticalAlignmentEnum

Namespace: TUIS.Components.Masks

Assembly: TUIS.dll

```
public enum TextMask.VerticalAlignmentEnum
```

## Fields

`Bottom = 2`

`Center = 1`

`Top = 0`

# Namespace TUIS.Systems

## Classes

InputSystem

TimeSystem

# Class InputSystem

Namespace: TUIS.Systems

Assembly: TUIS.dll

```
public class InputSystem : IDisposable
```

**Inheritance**

object ← InputSystem

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Methods

### Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

### Start()

```
public void Start()
```

## Events

### OnKeyPress

```
public event Action<ConsoleKeyInfo>? OnKeyPress
```

## Event Type

Action<ConsoleKeyInfo>

# Class TimeSystem

Namespace: TUIS.Systems

Assembly: TUIS.dll

```
public class TimeSystem
```

**Inheritance**

object ← TimeSystem

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

## Constructors

### TimeSystem()

```
public TimeSystem()
```

## Fields

### Timer

```
public readonly Timer Timer
```

#### Field Value

Timer

## Properties

### MiliSecond

```
public byte MiliSecond { get; }
```

### Property Value

byte

## Methods

### AddTimedEvent(ElapsedEventHandler?)

```
public void AddTimedEvent(ElapsedEventHandler? eventHandler)
```

### Parameters

`eventHandler` ElapsedEventHandler

### Start()

```
public void Start()
```